МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для оптимизации логистических маршрутов службы экспресс-доставки»

Выполнил: Иванов Иван Иванович
Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /
«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Набиуллину Артуру Фанилевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления проектами и отслеживания задач»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	05.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	11.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	25.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования)	7	02.05.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	09.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Реализация подсистем аутентификации и авторизации пользователей	8	21.05.2026	
Задача 3.2. Реализация системы распределения ролей и проверки прав доступа к функциям приложения	7	26.05.2026	
Задача 3.3. Реализация CRUD-интерфейсов для управления проектами, задачами и профилями	7	30.05.2026	
Задача 3.4. Разработка функционала загрузки и скачивания файлов, прикрепленных к задачам	12	03.06.2026	
Задача 3.5. Реализация модуля агрегирования данных для расчёта и вывода статистики по проектам	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	09.06.2026	
Задача 4.2. Деплой приложения на хостинг	4	11.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания « 31 » марта 2026 г.

Дата сдачи выполненной работы « 13 » июня 2026 г.

Задание принял к исполнению

« 31 » марта 2026 г. _____ А. Ф. Набиуллин
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ **6**

1.1 Обзор существующих программных продуктов по теме работы 6

1.2 Анализ программных инструментов разработки веб-приложений 6

1.3 Формулировка цели и задач работы 7

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ **9**

2.1 Анализ целевой аудитории 9

2.2 Описание функциональности приложения 9

2.3 Проектирование модели данных 9

2.4 Разработка макетов страниц 11

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ **12**

3.1 Разработка базовой структуры приложения и вёрстка 12

3.2 Реализация модуля авторизации 12

3.3 Интеграция картографических сервисов 12

ЗАКЛЮЧЕНИЕ **14**

ВВЕДЕНИЕ

Актуальность темы. В современных условиях цифровизации бизнеса и повсеместного перехода на удаленный или гибридный формат работы, эффективное управление проектами становится критически важным фактором успеха любой организации. Использование традиционных методов учета задач (электронные таблицы, мессенджеры) приводит к потере информации, срыву сроков и снижению производительности команды. Разработка специализированных веб-приложений для трекинга задач позволяет централизовать рабочие процессы, автоматизировать контроль поручений и обеспечить прозрачность работы на всех этапах жизненного цикла проекта.

Степень разработанности проблемы. На рынке существует множество систем управления проектами (Jira, Trello, Asana и др.). Однако большинство корпоративных решений обладают избыточным функционалом, сложны в освоении и имеют высокую стоимость лицензий, что делает их нерентабельными для малого бизнеса, студенческих команд или индивидуального использования. В связи с этим, создание легковесного, интуитивно понятного веб-приложения с базовым, но достаточным набором функций (CRUD-операции, ролевая модель, прикрепление файлов) остается актуальной практической задачей.

Объект исследования — процессы управления проектами, распределения задач и контроля их выполнения в рабочих группах.

Предмет исследования — программные средства, алгоритмы и методы веб-разработки, применяемые для создания систем отслеживания задач (Task Trackers).

Целью данной работы является проектирование и реализация клиент-серверного веб-приложения, предназначенного для эффективного управления проектами и отслеживания статусов выполнения задач с учетом ролевой модели доступа.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов по теме работы

На современном рынке программного обеспечения для управления проектами можно выделить несколько лидеров. Для определения требований к разрабатываемой системе проведем сравнительный анализ популярных платформ. Результаты сравнения представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ систем управления проектами

Название	**Ключевые особенности**	**Недостатки (для малых команд)**
Jira	Мощный инструментарий для Agile-разработки, детальная настройка рабочих процессов, глубокая интеграция с CI/CD.	Перегруженный интерфейс, высокая сложность первоначальной настройки, ресурсоемкость.
Trello	Интуитивно понятный интерфейс на базе Kanban-досок, простота использования, визуализация процессов.	Ограниченные возможности для сложной аналитики и агрегации данных, слабый контроль доступа в бесплатной версии.
Asana	Гибкое представление задач (списки, доски, таймлайны), удобное управление зависимостями между задачами.	Строгие ограничения в базовом тарифе, навязчивое предложение платных подписок.

Анализ показал, что существующие решения либо слишком сложны и дороги (Jira), либо ограничивают важный функционал, такой как детальная статистика или расширенные права доступа, в базовых версиях (Trello, Asana). Это подтверждает целесообразность разработки собственного легковесного решения, объединяющего простоту интерфейса и наличие необходимых инструментов управления.

1.2 Анализ программных инструментов разработки веб-приложений

Для реализации проекта выбран подход создания веб-приложения на базе языка программирования Python. Этот выбор обусловлен высокой скоростью разработки, читаемостью кода и богатой экосистемой библиотек.

- **Frontend-часть:** Пользовательский интерфейс будет реализован с использованием классических веб-технологий (HTML5, CSS3, JavaScript). Для рендеринга динамического контента на стороне сервера выбран шаблонизатор **Jinja2**, тесно интегрированный во Flask.

- **Backend-фреймворк:** В качестве основы серверной части выбран микрофреймворк **Flask**. В отличие от тяжеловесных решений (таких как Django), Flask предоставляет гибкость архитектуры, позволяя разработчику самостоятельно выбирать необходимые модули. Он отлично подходит для реализации маршрутизации, обработки HTTP-запросов, а также интеграции подсистем аутентификации (например, через Flask-Login).

- **Работа с базой данных:** Для взаимодействия с базой данных используется **SQLAlchemy** — мощная библиотека объектно-реляционного отображения (ORM). Использование ORM позволяет описывать структуру таблиц (Пользователи, Проекты, Задачи) в виде классов Python, избегая написания чистых SQL-запросов. SQLAlchemy обеспечивает безопасность (защиту от SQL-инъекций), поддержку CRUD-операций и удобные инструменты для агрегации данных (расчета статистики по проектам).

1.3 Формулировка цели и задач работы

На основе проведенного анализа предметной области (п. 1.1) и выбора технологического стека (п. 1.2), главная **цель работы** заключается в создании веб-приложения для управления проектами и отслеживания задач на базе фреймворка Flask.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ целевой аудитории и спроектировать пользовательские сценарии (Use Cases);

2. Спроектировать логическую и физическую модели реляционной базы данных;

3. Разработать макеты пользовательского интерфейса (Wireframes);

4. Реализовать подсистему аутентификации пользователей и настроить систему распределения ролей;
5. Разработать программные модули (CRUD-интерфейсы) для управления сущностями «Проект», «Задача» и «Пользователь»;
6. Реализовать функционал загрузки и скачивания файлов, прикрепленных к задачам;
7. Разработать модуль агрегирования данных для вывода аналитической статистики по проектам;
8. Развернуть готовое веб-приложение на удаленном сервере (хостинге).

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемой системы являются сотрудники логистических компаний. Выделяются две ключевые роли:

- **Диспетчер-логист** — сотрудник, ответственный за распределение заказов и контроль их выполнения в реальном времени.
- **Курьер** — конечный исполнитель, использующий мобильную версию приложения для навигации по маршруту.

Для каждой роли проведено профилирование по методу персонажей (Personas) для минимизации когнитивной нагрузки на интерфейс.

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователя с системой. Основной бизнес-кейс представлен в виде *User Story*: «Я, как диспетчер, хочу импортировать список адресов, чтобы система автоматически построила кратчайший маршрут для 5 курьеров».

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case).

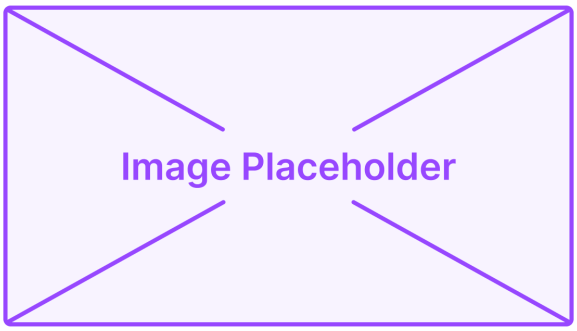

Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

При выборе архитектуры базы данных был проведен сравнительный анализ эффективности различных решений, опирающийся на данные о произ-

водительности современных СУБД [1].

Структура таблицы заказов представлена в табл. 2.1.

Таблица 2.1 – Структура таблицы заказов (`orders`)

Поле	**Тип**	**Ограничение**	**Описание**
id	UUID	PRIMARY KEY	Уникальный идентификатор заказа
customer_id	UUID	FOREIGN KEY	Ссылка на таблицу клиентов
address	TEXT	NOT NULL	Полный адрес доставки
lat	NUMERIC	NOT NULL	Широта точки (WGS-84)
lng	NUMERIC	NOT NULL	Долгота точки (WGS-84)
weight	FLOAT	CHECK > 0	Вес груза, кг
volume	FLOAT	NOT NULL	Объем груза, $м^3$
time_from	TIME	NOT NULL	Начало окна доставки
time_to	TIME	NOT NULL	Конец окна доставки
priority	INT	DEFAULT 1	Приоритет (1–5)
status	VARCHAR	’pending’	Статус (ожидание, в пути, выполнен)
courier_id	UUID	NULL	Назначенный исполнитель
created_at	TIMESTAMP	DEFAULT NOW()	Время создания записи
updated_at	TIMESTAMP	DEFAULT NOW()	Время последнего изменения
is_fragile	BOOLEAN	FALSE	Хрупкий груз
contact_phone	VARCHAR	NOT NULL	Телефон получателя
price	MONEY	NOT NULL	Стоимость доставки
external_id	VARCHAR	UNIQUE	ID из внешней системы (1C/ERP)

Продолжение таблицы 2.1

Поле	**Тип**	**Ограничение**	**Описание**
notes	TEXT	NULL	Комментарий для курьера
attempts	INT	DEFAULT 0	Количество попыток доставки

Общая стоимость маршрута для службы доставки рассчитывается следующим образом:

$$C_{total} = \sum_{i=1}^{n} (L_i \cdot P_{fuel} + T_i \cdot W_{hour}) + S_{fixed}, \tag{1}$$

где

L_i — протяженность i-го участка маршрута, км;

P_{fuel} — стоимость топлива на 1 км пути;

T_i — время в пути на участке;

W_{hour} — почасовая ставка оплаты труда курьера;

S_{fixed} — постоянные издержки на обслуживание автопарка.

2.4 Разработка макетов страниц

Макеты страниц

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка

В соответствии с задачей 3.1, была реализована компонентная структура фронтенд-части на базе библиотеки **React.js**. Использование хуков (*Hooks*) позволило эффективно управлять состоянием формы заказа. Пример базового компонента представлен в листинге 1.

Листинг 1 – Пример структуры компонента карточки заказа

```
// Комментарий
const OrderCard = ({ order }) => {
  return (
    <div className="order-card">
      <h3Заказ> #{order.id}</h3>
      <pАдрес>: {order.address}</p>
      <button onClick={() => addToRoute(order.id)}>
        Добавить в маршрут
      </button>
    </div>
  );
};
```

3.2 Реализация модуля авторизации

Согласно задаче 3.2, в системе реализована ролевая модель доступа. Для обеспечения безопасности используется протокол **JWT** (JSON Web Token). При входе в систему сервер генерирует токен, содержащий поле `role`, которое может принимать значения `'dispatcher'` или `'courier'`.

3.3 Интеграция картографических сервисов

Ключевым этапом разработки (задача 3.4) стала интеграция API **OpenStreetMap** через библиотеку *Leaflet*. Для решения транспортной задачи был выбран жадный алгоритм с последующей оптимизацией методом локального поиска.

Данные о маршруте передаются в формате JSON (см. листинг 2).

Как видно из листинга 2, каждая точка содержит порядковый номер (`sequence`), что позволяет курьеру следовать по оптимальному пути, рассчитанному на сервере.

Листинг 2 – Структура данных оптимизированного маршрута

```
{
  "route_id": "r-101",
  "courier_id": "u-45",
  "waypoints": [
    {"lat": 55.75, "lng": 37.61, "sequence": 1},
    {"lat": 55.80, "lng": 37.70, "sequence": 2}
  ],
  "total_distance_km": 12.4
}
```

ЗАКЛЮЧЕНИЕ

В результате проведенного исследования подтверждена гипотеза о том, что автоматизация логистических процессов на базе специализированного веб-приложения позволяет существенно повысить эффективность работы служб экспресс-доставки. Теоретический анализ показал, что классическая задача маршрутизации транспортных средств (VRP) в современных условиях требует адаптации под динамические изменения городского трафика и специфику «последней мили».

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Использование открытых ГИС-технологий и компонентного подхода (React/Node.js) обеспечивает необходимую архитектурную гибкость, позволяя интегрировать алгоритмы оптимизации без зависимости от проприетарных платных сервисов.

2. Проектирование модели данных с поддержкой пространственных расширений (PostGIS) является критическим условием для масштабируемости системы, так как позволяет эффективно обрабатывать геозапросы в режиме реального времени.

3. Внедрение эвристических методов построения маршрутов демонстрирует устойчивый экономический эффект, выраженный в снижении совокупного пробега автотранспорта и сокращении времени планирования рейсов диспетчером.

Рекомендации и предложения. Для практического применения разработанного решения в деятельности малых логистических компаний предлагается использовать модель облачного развертывания. Это обеспечит бесперебойный доступ к интерфейсу управления и мобильному клиенту курьера при минимальных затратах на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счет внедрения методов предиктивной аналитики для прогнозирования времени прибытия (ETA) с учетом исторических данных о заторах, а также разработки модуля динамиче-

ской корректировки маршрута при поступлении срочных заявок в процессе выполнения рейса.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [2], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

[1] Петров, П. П. Анализ производительности современных СУБД [Текст] / П. П. Петров, С. С. Сидоров // Вестник информационных технологий. — 2024. — № 2. — С. 15–22.

[2] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.